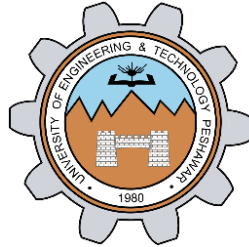


Traffic Light Controller using FSM

LAB # 12



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

2nd June 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives:

Implement a Traffic Light controller using FSM.

Software used:

Xilinx ISE

Introduction

The lab this week will continue the introduction to FSMs with a simple Traffic Light Controller design project. In the lab, you are required to create a state diagram of a Mealy machine, which implements a traffic light controller, based on provided guidelines. You will then use this state diagram to write the behavioral Verilog description of the traffic light controller. The testing of your traffic light controller will take place during the lab session using the ISE development tools and the Spartan 6 board. As with the previous lab, we will make use of the character LEDs to display the outputs of our digital circuit.

The Traffic Light Controller

This week you will design a traffic light controller for the highway and farm road intersection shown in Figure 1. For simplicity, we will assume that the traffic lights on opposite ends of the intersection are the same; in other words, if the light for the North bound traffic is green, then the light for the South bound traffic is also green. The traffic light controller outputs two 2-bit signals, highwaySignal and farmSignal, for the highway road and the farm road, respectively.

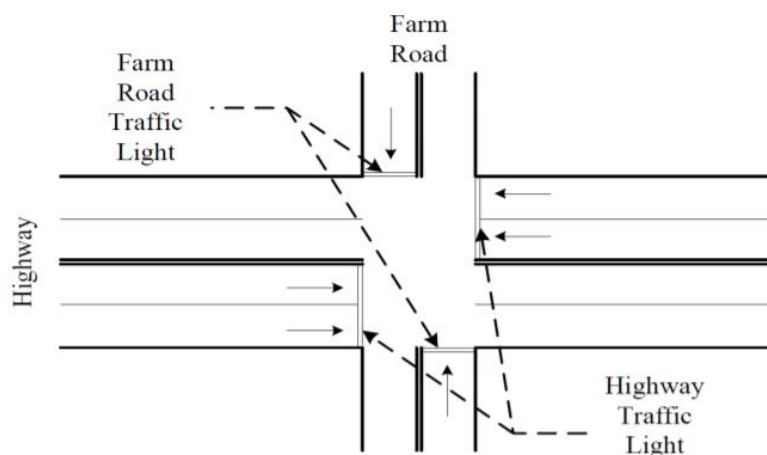


Figure 1: Example Illustration

The following encoding is used for both signals:

001 : Green

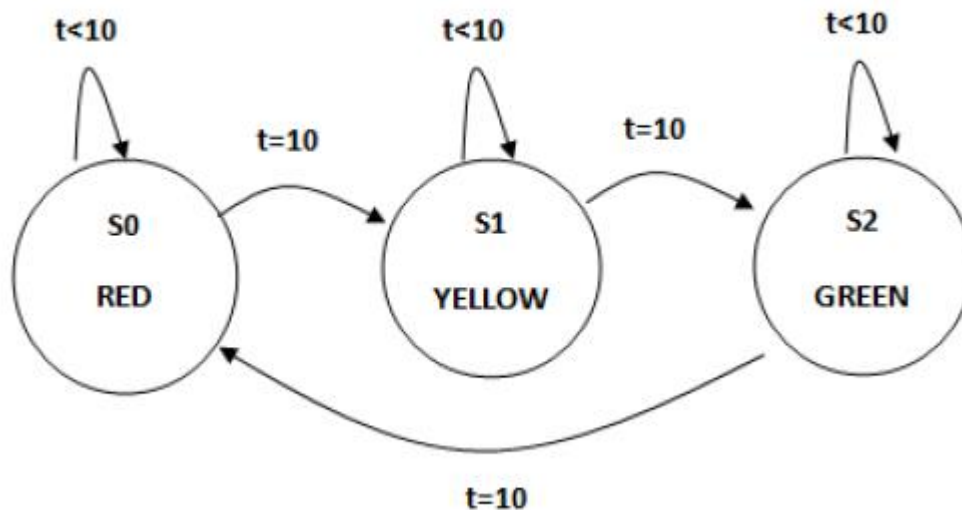
010 : Yellow

100 : Red

Obviously, to avoid accidents, the highway lights and farm road lights must not be green at the same time. Furthermore, the transitions from green on each road must be followed by 3 seconds of yellow. Because the traffic on the highway is much greater than that of the farm road, the traffic light must remain green on the highway longer than on the farm road. For this design, the highway light must remain green unless there is a vehicle on the farm road, while the farm road light must remain green for only 10 seconds.

The brain of the traffic light controller is the FSM. For the initial design, Vehicle, Clock and Rst constitute the only input to the FSM. The output of the FSM includes the highwaySignal and farmSignal. The operation of the FSM can be described by using a state diagram as shown in figure 2.

Figure 1. Task1:



Task 2:

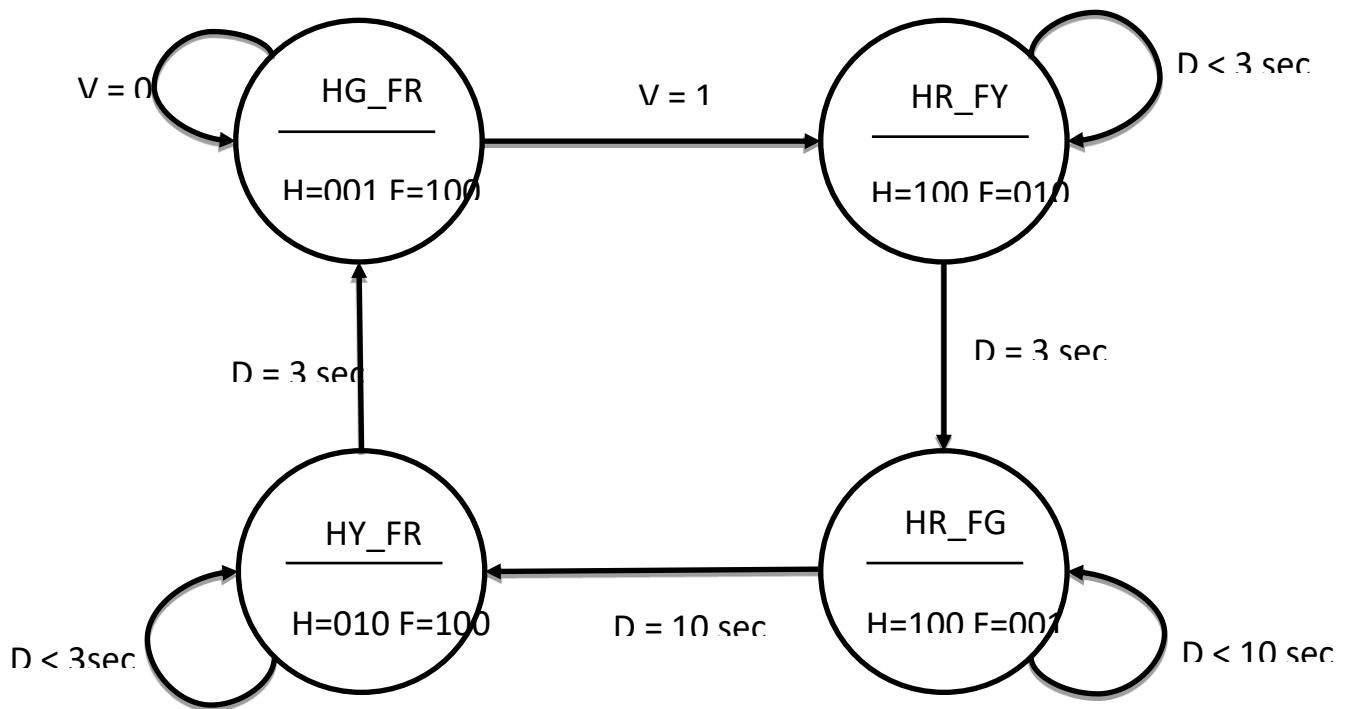


Figure 2: State Transition Diagram

Lab Task 1:

Code:

```

1 module top(CLK_IN, RST, OUT_LED, OUT_SEG);
2   input CLK_IN, RST;
3   output [2:0]OUT_LED;
4   output [7:0]OUT_SEG;
5   wire [2:0]OUT_LED_WIRE;
6   wire CLK;
7
8   clock_divider divider(CLK_IN, CLK);
9   traffic_controller dut(CLK, RST, OUT_LED_WIRE);
10  BCD_to_SevenSeg bcd_dec({1'b0, OUT_LED_WIRE}, OUT_SEG);
11
12  assign OUT_LED = OUT_LED_WIRE;
13 endmodule

```

Figure 1.1 top module

```

1 module clock_divider(
2     input clk_in,
3     output reg clk_out
4 );
5     reg [29:0] counter = 30'd0;
6     parameter divisor = 30'd1000000000;
7
8     always @(posedge clk_in) begin
9         counter <= counter + 28'd1;
10
11         if (counter >= (divisor - 1))
12             counter <= 30'd0;
13
14         clk_out <= (counter < divisor/2) ? 1'b1 : 1'b0;
15     end
16 endmodule

```

Figure 1.2: Clock Divider Code

```

21 module traffic_controller(clk, rst, out);
22     input clk, rst;
23     output reg [2:0] out;
24
25
26     reg [2:0] PS, NS;
27     parameter S0 = 3'b001, S1 = 3'b010, S2 = 3'b011;
28
29
30     always@(posedge clk)
31         if(rst)
32             PS = S0;
33         else
34             PS = NS;
35
36     always@(PS)
37         case(PS)
38             S0: begin
39                 NS = S1;

```

Design Summary (out of date) x Top.v x clock_divider.v x traffic_controller_fsm.v x

Figure 1.3.1: Traffic Controller FSM Code – Part 1

```
39         NS = S1;
40         out = S0;
41     end
42
43     S1: begin
44         NS = S2;
45         out = S1;
46     end
47
48     S2: begin
49         NS = S0;
50         out = S2;
51     end
52
53     default: begin
54         NS = S0;
55         out = S0;
56     end
57 endcase
58 endmodule
```

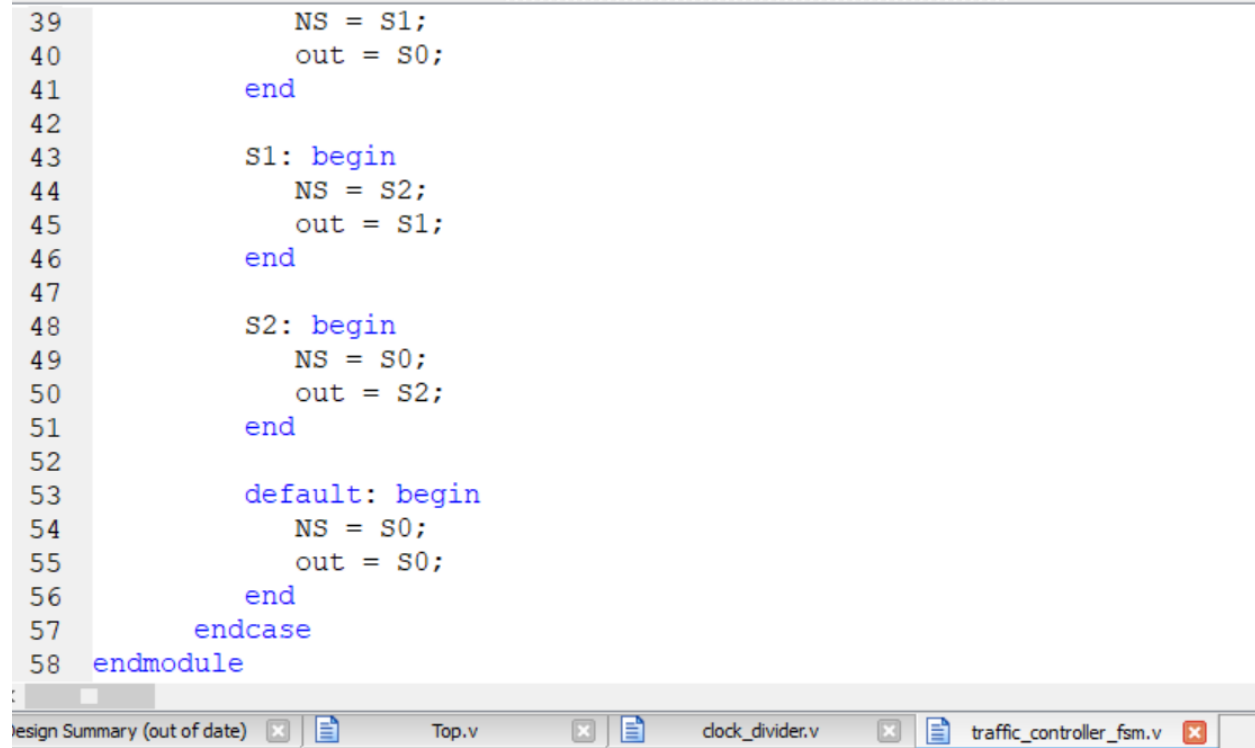


Figure 1.3.2: Traffic Controller FSM Code – Part 2

```
1 module BCD_to_SevenSeg(
2     input [3:0] bcd,
3     output reg [7:0] o
4 );
5     always @(*)
6         case(bcd) //gfedcba
7             4'b0000: o = 8'b1000000;
8             4'b0001: o = 8'b11111001;
9             4'b0010: o = 8'b10100100;
10            4'b0011: o = 8'b10110000;
11            4'b0100: o = 8'b10011001;
12            4'b0101: o = 8'b10010010;
13            4'b0110: o = 8'b10000010;
14            4'b0111: o = 8'b11111000;
15            4'b1000: o = 8'b10000000;
16            4'b1001: o = 8'b10010000;
17            default: o = 8'b11111111;
18        endcase
19
20 endmodule
```

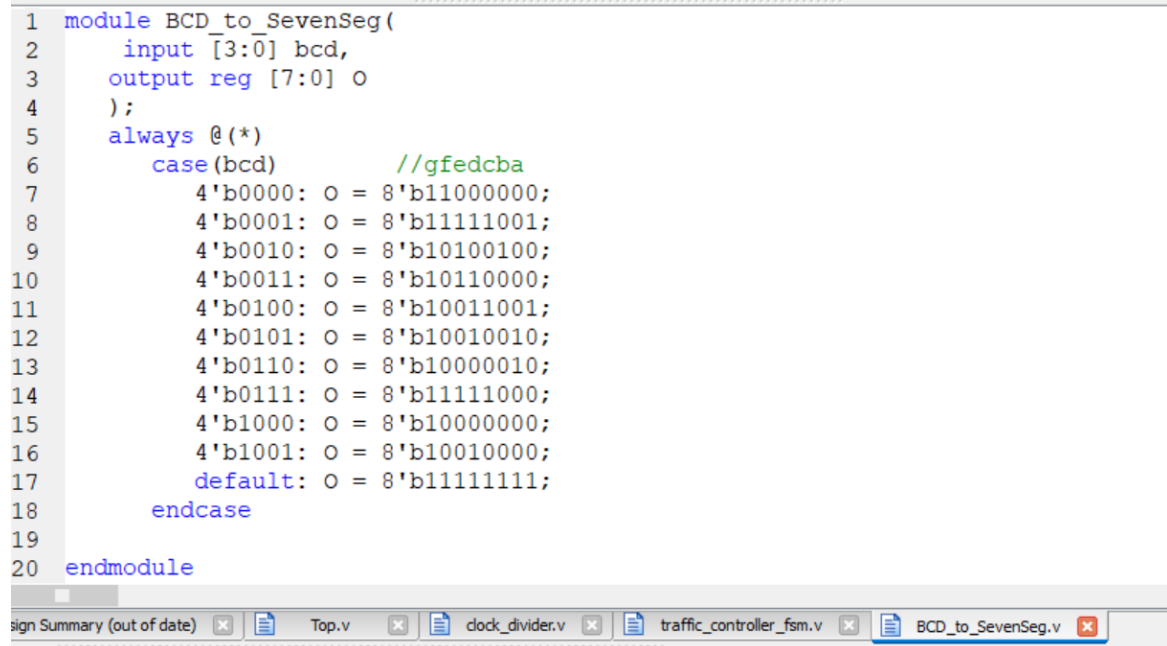


Figure 1.4: Binary Coded Decimal (BCD) to Seven Segment Decoder

```

k_generator.v  ADD.ucf  ADD.v  fp.v  Top.ucf  three_BCD.v  Top.ucf
CONFIG VCCAUX = "3.3" ;

NET "CLK_IN"          LOC = V10      | IOSTANDARD = LVCMOS33 | PERIOD = 100MHz ;
NET "RST"             LOC = C17      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULL

NET "OUT_LED[0]"      LOC = P15      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW =
NET "OUT_LED[1]"      LOC = P16      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW =
NET "OUT_LED[2]"      LOC = N15      | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW =

NET "OUT_SEG[0]"      LOC = A3        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[1]"      LOC = B4        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[2]"      LOC = A4        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[3]"      LOC = C4        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[4]"      LOC = C5        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[5]"      LOC = D6        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[6]"      LOC = C6        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
NET "OUT_SEG[7]"      LOC = A5        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;

NET "OUT_SEG[7]"      LOC = B2        | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;

```

Figure 1.5: UCF File

Output:

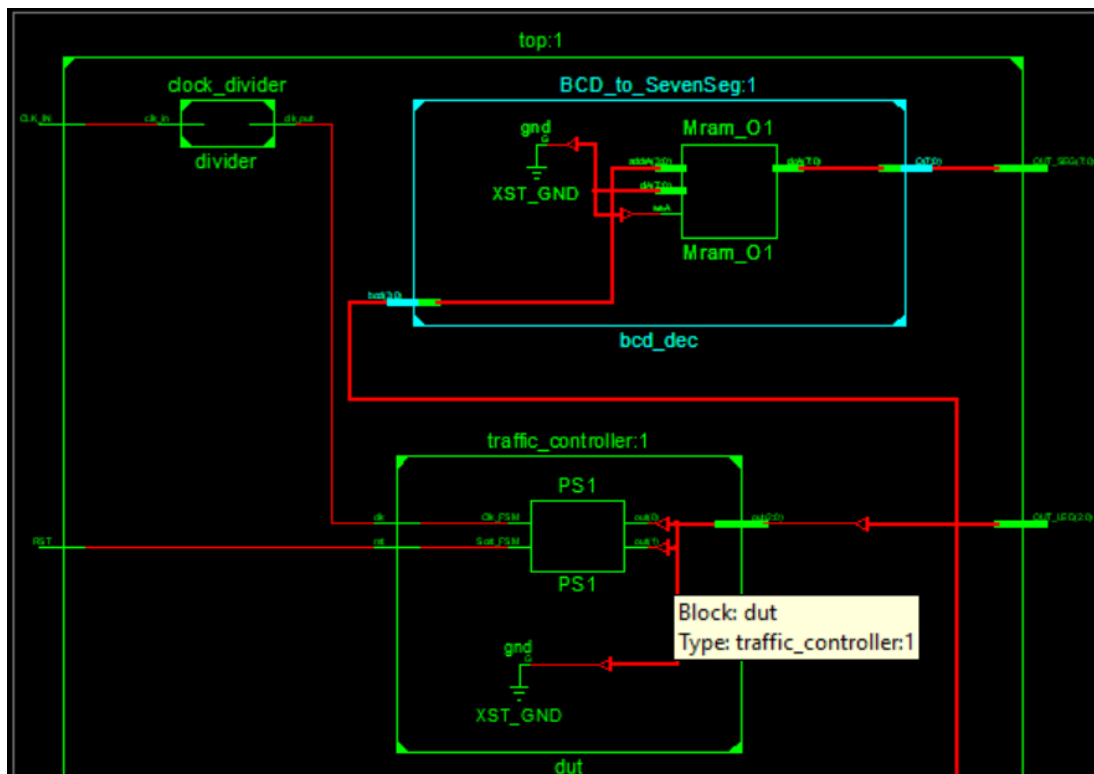


Figure 1.6: RTL Schematic of Traffic Light Controller Top Module

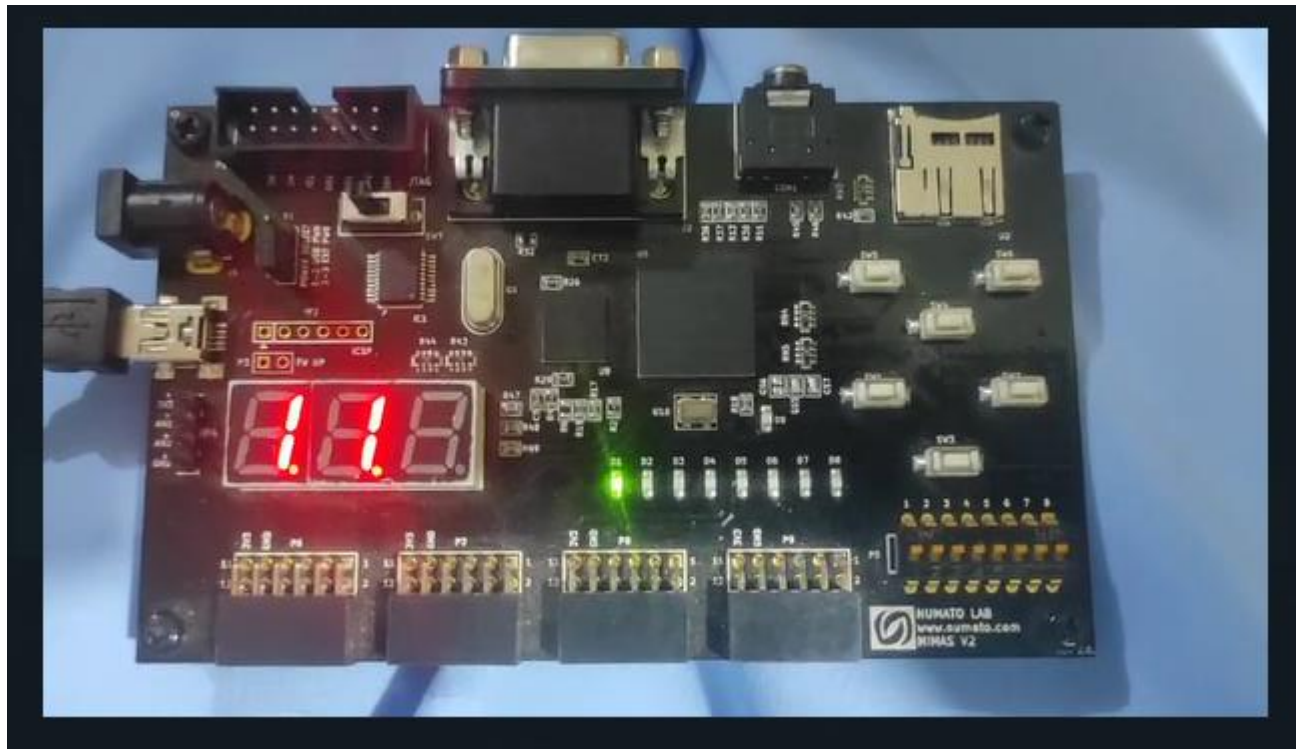


Figure 1.7: Task 1 Implementation on Spartan6 FPGA Board (State S0)

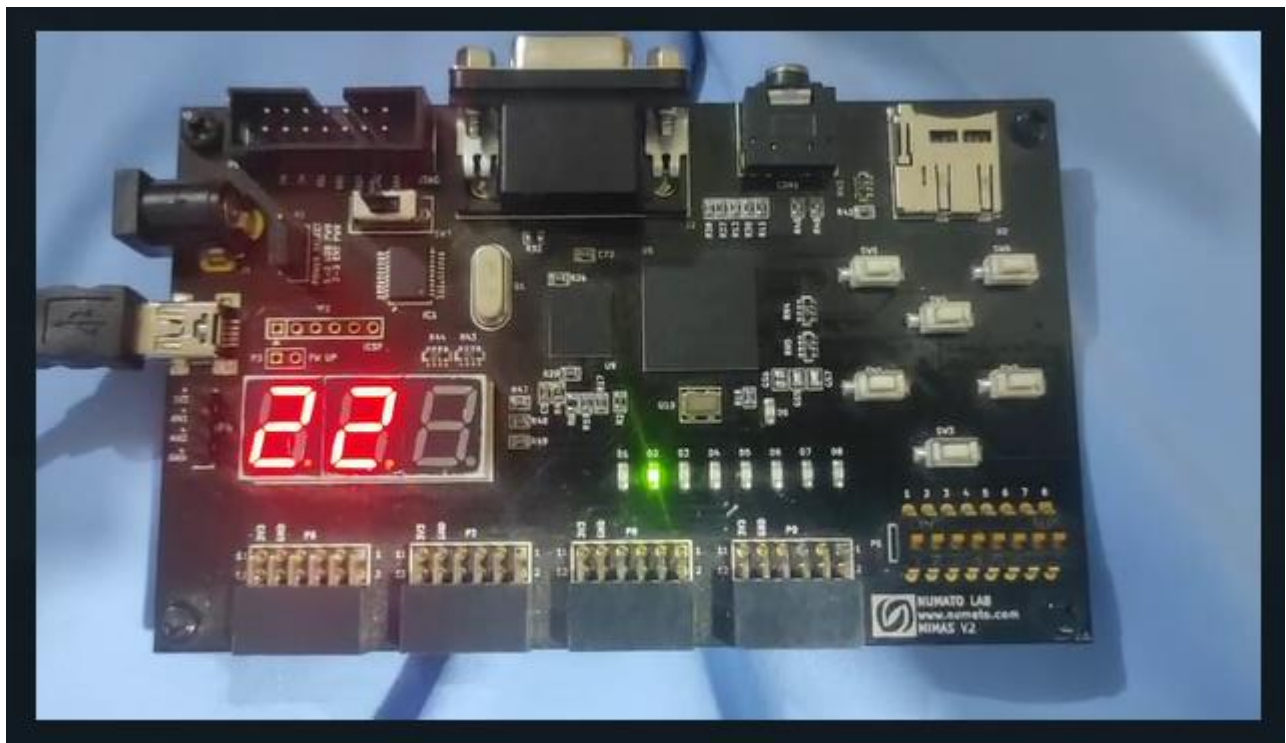


Figure 1.8: Task 1 Implementation on Spartan6 FPGA Board (State S1)

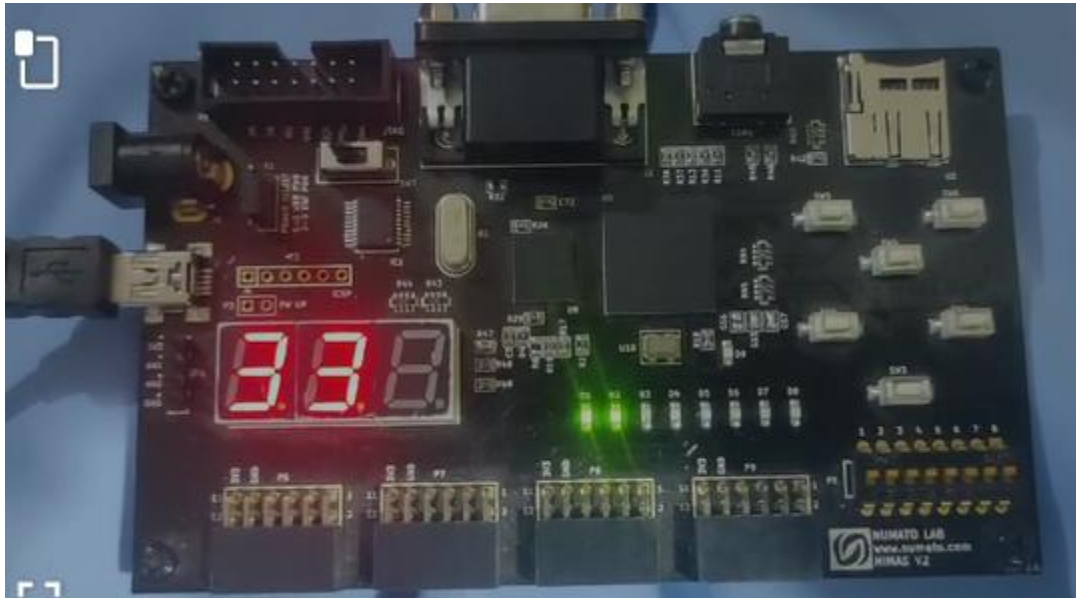


Figure 1.9: Task 1 Implementation on Spartan6 FPGA Board (State S2)

Lab Task 2:

Code:

```

1 module top(CLK_IN, RST, OUT_LED1, OUT_LED2, OUT_SEG);
2     input CLK_IN, RST;
3     output [2:0]OUT_LED1;
4     output [2:0]OUT_LED2;
5
6     output [7:0]OUT_SEG;
7
8     wire [3:0]COUNT;
9     wire CLK;
10
11     clock_divider divider(CLK_IN, CLK);
12     traffic_controller dut(CLK, RST, OUT_LED1, OUT_LED2, COUNT);
13     BCD_to_SevenSeg bcd_dec(COUNT,OUT_SEG);
14
15 endmodule
16
17

```

Figure 2.1 top module

```

21 module traffic_controller(clk, v, FR, HR, count);
22     input clk, v;
23     output reg [2:0]FR;
24     output reg [2:0]HR;
25     output [3:0]count;
26
27     reg [3:0]count;
28
29     reg [2:0]PS, NS;
30     parameter S0 = 3'b000, S1 = 3'b001, S2 = 3'b010, S3 = 3'b011 , S4 = 3'b100;
31     parameter GREEN = 3'b001, YELLOW = 3'b010, RED = 3'b100;
32
33
34     always@(posedge clk) begin
35         if(v)
36             PS <= S0;
37         else
38             PS <= NS;

```

Design Summary | Top.v | traffic_controller_fsm.v

Figure 2.2.1: Highway Traffic Controller FSM Code – Part 1

```

36         PS <= S0;
37     else
38         PS <= NS;
39
40     count <= (PS == S0)? 4'b0000 : count + 1;
41
42 end
43
44 always@(*)
45     case(PS)
46     S0: begin
47         NS = S1;
48         HR = GREEN;
49         FR = RED;
50     end
51
52     S1: begin
53         NS = (count < 2 ) ? S1 : S2;

```

Figure 2.2.2: Highway Traffic Controller FSM Code – Part 2

```
52         S1: begin
53             NS = (count < 2 ) ? S1 : S2;
54             HR = RED;
55             FR = YELLOW;
56         end
57
58         S2: begin
59             NS = (count < 9 ) ? S2 : S3;
60             HR = RED;
61             FR = GREEN;
62         end
63
64
65         S3: begin
66             NS = (count < 12 ) ? S3 : S0;;
67             HR = YELLOW;
68             FR = RED;
69         end
```

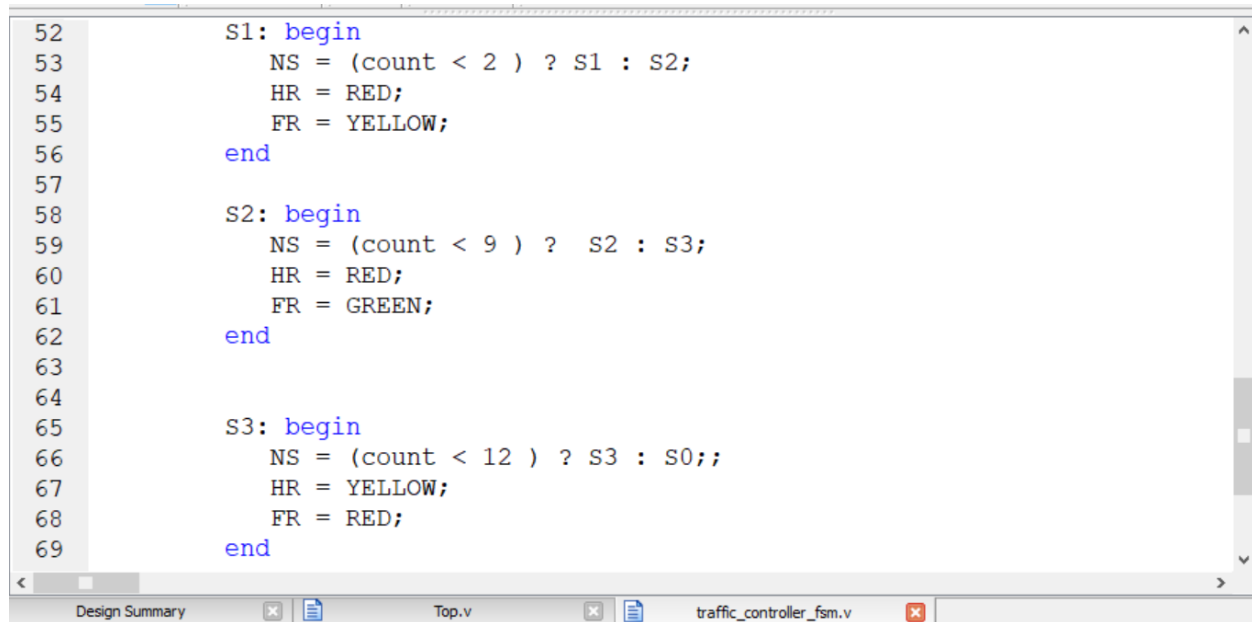


Figure 2.2.3: Highway Traffic Controller FSM Code – Part 3

```
69         end
70
71         default: begin
72             NS = S0;
73         end
74
75     endcase
76 endmodule
77
```

Figure 2.2.3: Highway Traffic Controller FSM Code – Part 4

```

1 CONFIG VCCAUX = "3.3" ;
2 NET "CLK_IN" LOC = V10 | IOSTANDARD = LVCMOS33 | PERIOD = 100MHz ;
3 NET "RST" LOC = C17 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST | PULL
4
5 NET "OUT_LED1[2]" LOC = P15 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
6 NET "OUT_LED1[1]" LOC = P16 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
7 NET "OUT_LED1[0]" LOC = N15 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
8 #NET "LED[4]" LOC = N16 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FA
9 #NET "LED[3]" LOC = U17 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FA
10 NET "OUT_LED2[2]" LOC = U18 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
11 NET "OUT_LED2[1]" LOC = T17 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
12 NET "OUT_LED2[0]" LOC = T18 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW
13
14 NET "OUT_SEG[0]" LOC = A3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
15 NET "OUT_SEG[1]" LOC = B4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
16 NET "OUT_SEG[2]" LOC = A4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
17 NET "OUT_SEG[3]" LOC = C4 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
18 NET "OUT_SEG[4]" LOC = C5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
19 NET "OUT_SEG[5]" LOC = D6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
20 NET "OUT_SEG[6]" LOC = C6 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
21 NET "OUT_SEG[7]" LOC = A5 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ;
22
23 NET "OUT_SEG[7]" LOC = B3 | IOSTANDARD = LVCMOS33 | DRIVE = 8 | SLEW = FAST ; #Enab

```

Figure 2.3: UCF File

Output:

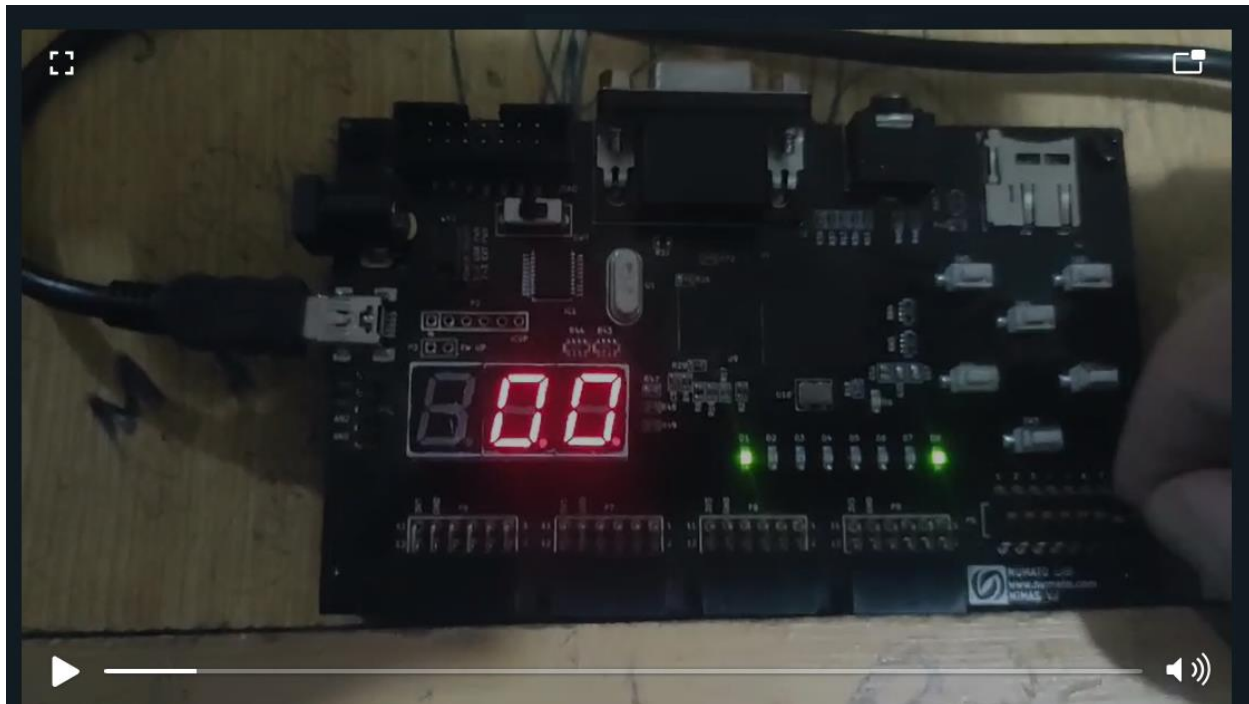


Figure 2.4: Task 2 Implementation on Spartan6 FPGA Board (State S0)

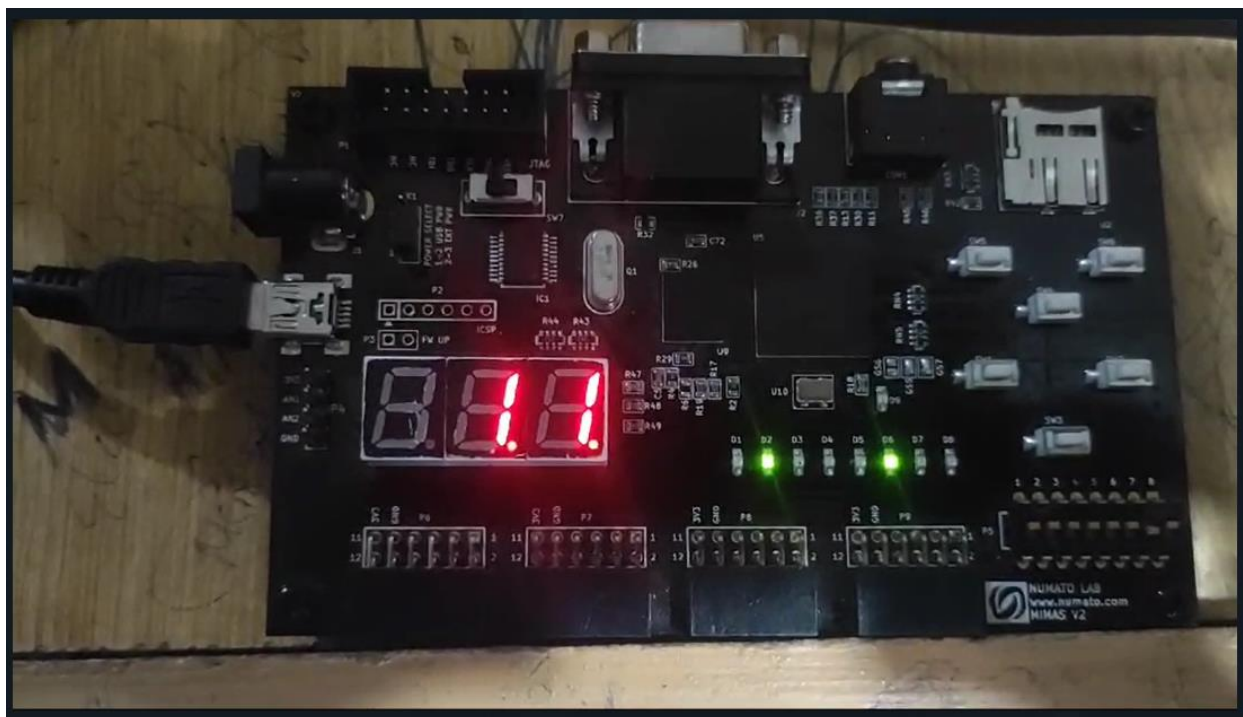


Figure 2.5: Task 2 Implementation on Spartan6 FPGA Board (State S1)

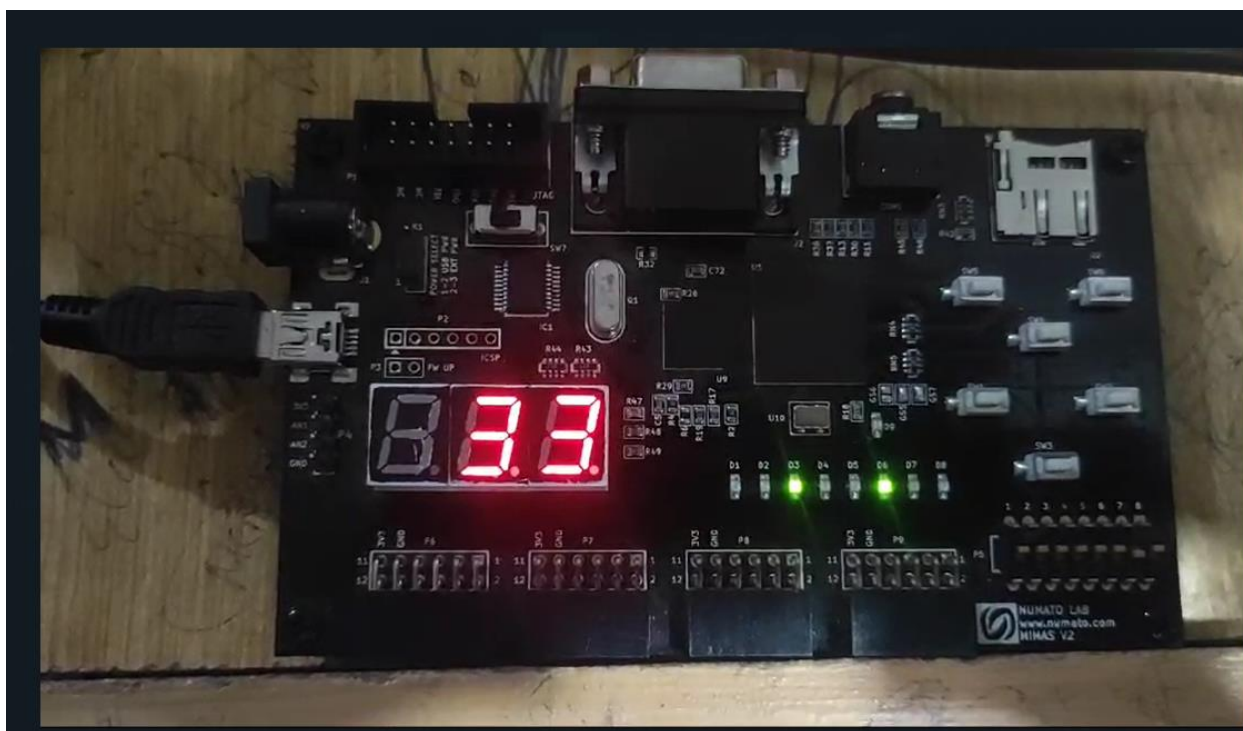


Figure 2.6: Task 2 Implementation on Spartan6 FPGA Board (State S2)

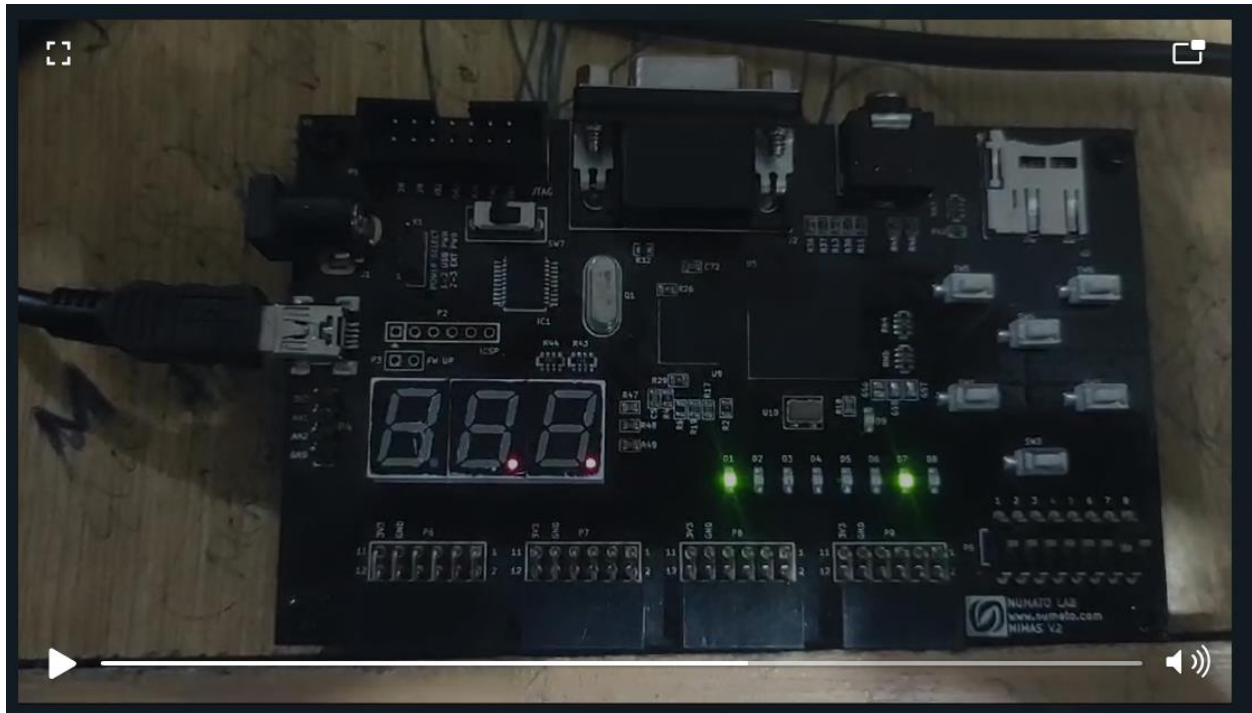


Figure 2.7: Task 2 Implementation on Spartan6 FPGA Board (State S3)